



同濟大學
TONGJI UNIVERSITY

计算机系统结构课程实验 总结报告

实验题目：动态流水线设计与性能定量分析

学号：2351520

姓名：毛星博

指导教师：陆有军

日期：2025 年 12 月 10 日

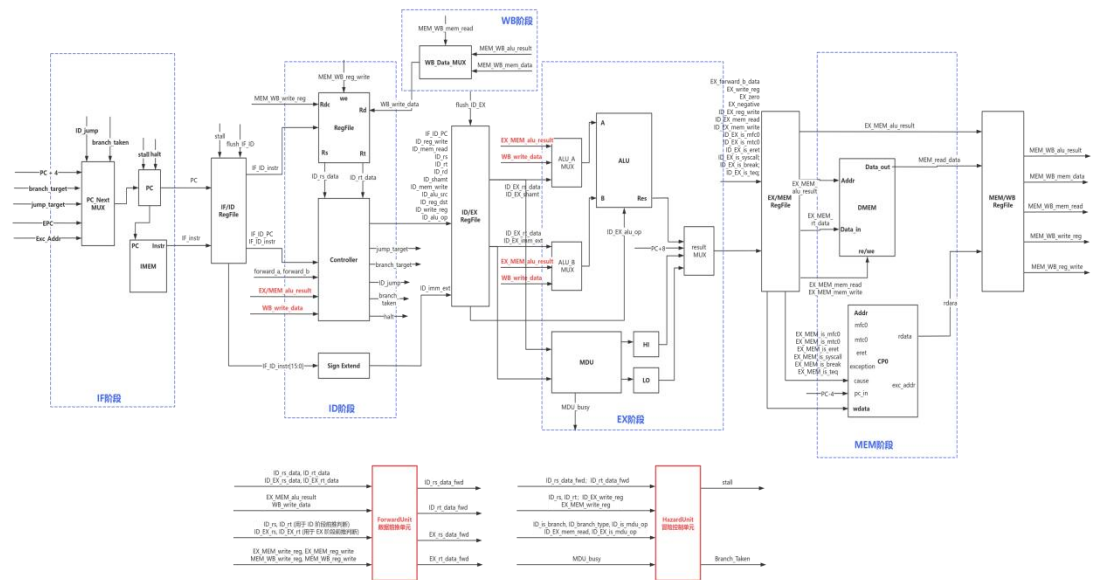
一、实验环境部署与硬件配置说明

- 操作系统环境：Windows 11 24H2 64 位
- 开发环境：vivado2016.2 + Modelsim
- 硬件配置：Xilinx NEXYS4 DDR Artix-7

二、实验的总体结构

在本次实验中，我完成了支持 54 条 MPIS 指令集的动态流水线 CPU 设计，并支持中断。在 CPU 运行验证程序的过程中，由按键或拨动开关产生一个暂停的中断，再次按键或拨动开关结束中断，继续运行后续的运算，并在数码管上动态显示运算值。

1、流水线 CPU 的总体架构图



2、54 条指令的选取，指令格式的设计

本次 CPU 的指令集基于 MIPS32 指令集架构，选取了其中最核心的 54 条指令进行实现。这些指令涵盖了算术运算、逻辑运算、移位操作、数据传输、分支跳转以及协处理器操作等多种类型，能够满足大多数复杂程序的运行需求。

指令列表如下：

序号	指令	格式	Op (31-26)	rs (25-21)	rt (20-16)	rd (15-11)	shamt (10-6)	funct (5-0)
算术								
1	add	R	0	rs	rt	rd	0	100000

序号	指令	格式	Op (31-26)	rs (25-21)	rt (20-16)	rd (15-11)	shamt (10-6)	funct (5-0)
2	addu	R	0	rs	rt	rd	0	100001
3	sub	R	0	rs	rt	rd	0	100010
4	subu	R	0	rs	rt	rd	0	100011
5	addi	I	1000	rs	rt	imm (16-bit)		
6	addiu	I	1001	rs	rt	imm (16-bit)		
7	clz	R	11100	rs	rt	rd	0	100000
8	mult	R	0	rs	rt	0	0	11000
9	multu	R	0	rs	rt	0	0	11001
10	div	R	0	rs	rt	0	0	11010
11	divu	R	0	rs	rt	0	0	11011
逻辑								
12	and	R	0	rs	rt	rd	0	100100
13	or	R	0	rs	rt	rd	0	100101
14	xor	R	0	rs	rt	rd	0	100110
15	nor	R	0	rs	rt	rd	0	100111
16	andi	I	1100	rs	rt	imm (16-bit)		
17	ori	I	1101	rs	rt	imm (16-bit)		
18	xori	I	1110	rs	rt	imm (16-bit)		
19	lui	I	1111	0	rt	imm (16-bit)		
移位								
20	sll	R	0	0	rt	rd	shamt	0
21	srl	R	0	0	rt	rd	shamt	10
22	sra	R	0	0	rt	rd	shamt	11
23	sllv	R	0	rs	rt	rd	0	100
24	srlv	R	0	rs	rt	rd	0	110
25	srav	R	0	rs	rt	rd	0	111
访存								
26	lw	I	100011	rs	rt	imm (16-bit)		
27	sw	I	101011	rs	rt	imm (16-bit)		
28	lb	I	100000	rs	rt	imm (16-bit)		
29	lbu	I	100100	rs	rt	imm (16-bit)		
30	sb	I	101000	rs	rt	imm (16-bit)		
31	lh	I	100001	rs	rt	imm (16-bit)		
32	lhu	I	100101	rs	rt	imm (16-bit)		
33	sh	I	101001	rs	rt	imm (16-bit)		
34	mfhi	R	0	0	0	rd	0	10000
35	mflo	R	0	0	0	rd	0	10010
36	mthi	R	0	rs	0	0	0	10001
37	mtlo	R	0	rs	0	0	0	10011
分支								
38	beq	I	100	rs	rt	imm (16-bit)		
39	bne	I	101	rs	rt	imm (16-bit)		
40	bgez	I	1	rs	1	imm (16-bit)		

序号	指令	格式	Op (31-26)	rs (25-21)	rt (20-16)	rd (15-11)	shamt (10-6)	funct (5-0)
41	slt	R	0	rs	rt	rd	0	101010
42	sltu	R	0	rs	rt	rd	0	101011
43	slti	I	1010	rs	rt	imm (16-bit)		
44	sltiu	I	1011	rs	rt	imm (16-bit)		
跳转								
45	j	J	10	address (26-bit)				
46	jal	J	11	address (26-bit)				
47	jr	R	0	rs	0	0	0	1000
48	jalr	R	0	rs	0	rd	0	1001
系统/协处理								
49	syscall	R	0	0	0	0	0	1100
50	break	R	0	0	0	0	0	1101
51	teq	R	0	rs	rt	0	0	110100
52	mfc0	R	10000	0	rt	rd	0	0
53	mtc0	R	10000	100	rt	rd	0	0
54	eret	R	10000	10000	0	0	0	11000

三、总体架构部件的解释说明

1、5 级动态流水线总体结构部件的解释说明

本次我设计的 CPU 采用 MIPS 五级动态流水线架构，支持 54 条 MIPS 指令。设计包含了 IF(取指)、ID(译码)、EX(执行)、MEM(访存)、WB(写回) 五个阶段。

动态流水线需要再硬件层面解决指令冲突问题。为了解决流水线中的冒险问题，CPU 中集成了动态冒险检测单元 (Hazard Unit) 和数据前推单元 (Forwarding Unit)，使得 CPU 能够在硬件层面自动处理数据冒险（通过前推或气泡）和控制冒险（通过分支预测或刷新），无需依赖编译器的静态调度，从而实现了高效的指令并行执行。

分支预测方面，采用了延迟槽技术来处理冒险。无论分支是否跳转，跟在分支或跳转指令之后的一条指令（即延迟槽指令）总是会被执行。相应的，编写汇编程序时，需要由程序员手动负责在这些指令后填充有效的指令或 nop 指令。

CPU 采用哈佛结构（事实上流水线 CPU 也只能采用哈佛结构），将指令存储空间和数据存储空间在物理上独立开来。指令存储器 (IMEM) 专用于存放程序指令，支持 CPU 在 IF 阶段读取指令。数据存储器 (DMEM) 专用于存放程序数据，支持 CPU 在 MEM 阶段进行读写操作。这种独立的存储器设计允许 CPU 在同一个时钟周期内同时完成“取指”和“访存”操

作，避免了冯·诺依曼结构中的访存冲突，显著提高了流水线的吞吐率。

流水线各阶段：

- **取指阶段 (IF)：根据程序计数器 (PC) 的值从指令存储器中取出当前指令**
 - 实现逻辑：PC 寄存器输出当前地址到 IMEM；IMEM 返回 32 位指令字 IF_instr；加法器计算 $PC_plus_4 = PC + 4$ ，为顺序执行做准备；多路选择器根据 HazardUnit 的 stall 信号决定是否更新 PC 和 IF/ID 流水线寄存器。
 - 包含模块：IMEM (指令存储器), PC (程序计数器)
- **译码阶段 (ID)：对指令进行译码生成控制信号，并处理分支跳转和冒险检测**
 - 实现逻辑：Controller 解析指令，生成 reg_write, alu_op, mem_read 等控制信号。根据指令中的 rs, rt 字段从 RegFile 读取源操作数。分支判断：提前在 ID 阶段完成分支判断（如 beq, bgez），计算跳转目标地址 branch_target。冒险检测：HazardUnit 检测 Load-Use 冒险和控制冒险，必要时生成 stall 信号暂停流水线或刷新 IF/ID 寄存器。
 - 包含模块：Controller (控制器), RegFile (寄存器堆), HazardUnit (冒险检测单元), ForwardUnit (前推单元- ID 阶段分支前推)
- **执行阶段 (EX)：完成算术逻辑运算、乘法运算或计算访存地址**
 - 实现逻辑：ALU 运算：ALU 根据 alu_op 对操作数进行计算。操作数来源于 ID 阶段传递的值，或通过 ForwardUnit 从 MEM、WB 阶段前推得到的最新数据。乘除法：MDU 独立处理 mult, div 等耗时指令，运算结果存入 HI/LO 寄存器。前推处理：ForwardUnit 比较 EX_rs/EX_rt 与 MEM/WB 阶段的目标寄存器号，解决 EX 阶段的数据冒险。
 - 包含模块：ALU (算术逻辑单元), MDU (乘除法单元), ForwardUnit (前推单元- EX 阶段前推)
- **访存阶段 (MEM)：访问数据存储器进行数据读取或写入，并处理协处理器相关操作**
 - 实现逻辑：读写内存：如果是 Load/Store 指令，DMEM 根据 EX 阶段计算的地址进行读写。支持字、半字、字节访问。异常处理：CPO 模块在此阶段检测异常（如 syscall, break, teq），处理中断使能，并保存 EPC。
 - 包含模块：DMEM (数据存储器), CPO (协处理器 0)。
- **写回阶段 (WB)：将指令执行结果写回通用寄存器组**
 - 实现逻辑：根据指令类型，通过多路选择器确定写入数据来源。数据可能

来自 ALU 运算结果、DMEM 读出数据 (mem_data) 或 CP0 寄存器 (cp0_data)。

若 reg_write 信号有效, 将选定的 32 位数据写入 write_reg 指定的目标寄存器。

- 包含模块: RegFile (寄存器堆)

2、5 级动态流水线各个结构部件的解释说明

2.1 IF 阶段

IF 阶段主要负责根据程序计数器 (PC) 的值从指令存储器 (IMEM) 中读取当前指令, 并计算下一条指令的地址。实现了完整的 PC 更新逻辑, 能够根据异常、ERET 指令、分支跳转指令的优先级自动选择下一跳 PC 值, 同时支持由 HazardUnit 控制的流水线暂停 (Stall)。

```
// 下一跳 PC 选择优先级: 异常 > eret > jr > jump > branch > PC+4
// PC 异常处理向量地址 exception > eret > jr > jump > branch > PC+4
wire exc_occur = CP0_exception && !stall;
wire eret_occur = EX_MEM_is_eret && !stall;

assign PC_next = exc_occur    ? 32'h00400004 :
                  eret_occur  ? CP0_exc_addr :
                  ID_is_jr     ? ID_rs_data_fwd :
                  ID_is_jump   ? jump_target :
                  branch_taken ? branch_target :
                  PC_plus_4;

// PC 寄存器更新
always @(posedge clk or posedge rst) begin
    if (rst) begin
        PC <= 32'h00400000;
    end
    else if (ena && !stall) begin
        PC <= PC_next;
    end
    // stall 或 !ena 时 PC 保持不变
end
```

(1) IMEM 模块

指令存储器, 基于 IP 核 dist_mem_gen_0 实现。输入为 32 位 PC 地址, 输出对应地址处的 32 位指令机器码。将 PC 映射到存储器地址空间 (PC - 0x00400000), 并按字寻址读取指令。

```
module IMEM(
    input [31:0] PC,
    output [31:0] Instr
```

```

);

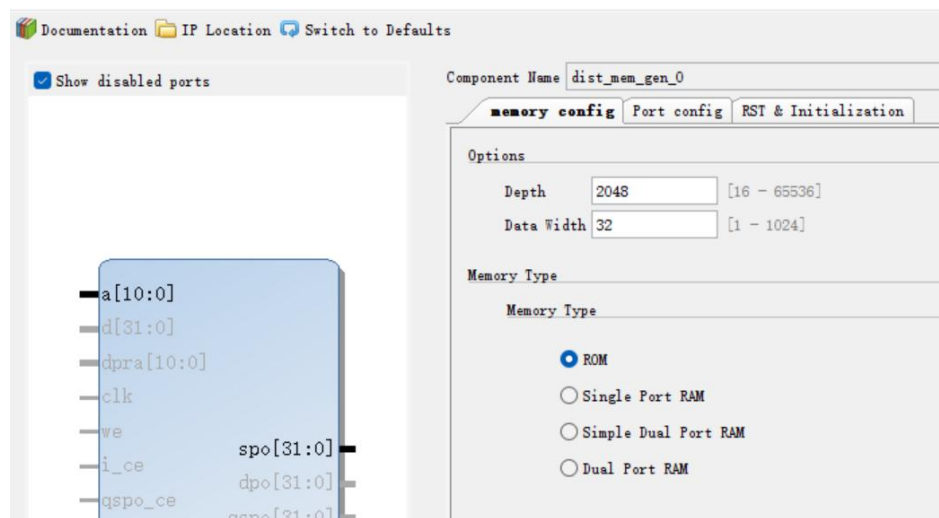
wire [10:0] PC_addr; // 11 位指令码地址
assign PC_addr = PC[12:2] - 32'h00400000; // 计算 PC 地址映射

// 实例化 IP 核，输入指令码地址返回对应的指令
dist_mem_gen_0 Instr_mem(
    .a(PC_addr),
    .spo(Instr)
);

endmodule

```

Distributed Memory Generator (8.0)



(2) IF_ID 寄存器

连接 IF 和 ID 阶段的流水线寄存器。在时钟上升沿保存当前的 PC 值 (IF_ID_PC) 和指令 (IF_ID_instr)。支持 stall 保持（暂停流水线时保存原值）和同步复位。

```

// 如果 stall 则保持不变，如果有 flush 信号(如分支预测失败)则清空
always @(posedge clk or posedge rst) begin
    if (rst) begin
        IF_ID_PC    <= 32'h00400000;
        IF_ID_instr <= 32'h00000000; // NOP
    end
    else if (ena && !stall) begin
        IF_ID_PC    <= PC;
        IF_ID_instr <= IF_instr;
    end
    // stall 或 !ena 保持不变
end

```

2.2 ID 阶段

ID 阶段通过调用 Controller 模块对指令进行译码，读取通用寄存器组，并生成后续阶段所需的控制信号。ID 阶段集成了提前分支判断和冒险检测逻辑，能够利用前推数据在 ID 阶段直接计算分支条件，从而减少控制冒险带来的开销。

(1) Controller 模块

主控制器，根据指令的 opcode 和 funct 字段进行译码。输出寄存器写使能 (reg_write)、ALU 操作码 (alu_op)、访存控制信号以及各类指令类型标识（如 is_branch, is_mdu_op 等），指导后续阶段的操作。

```
module Controller(  
    // ===== 输入信号 =====  
    input [31:0] instr,           // 32 位指令  
  
    // ===== 指令译码输出 =====  
    output [5:0] opcode,          // 操作码  
    output [5:0] funct,          // 功能码(R 型)  
    output [4:0] rs,             // 源寄存器 1  
    output [4:0] rt,             // 源寄存器 2  
    output [4:0] rd,             // 目的寄存器(R 型)  
    output [4:0] shamt,          // 移位量  
    output [15:0] imm,           // 立即数  
    output [25:0] addr,          // 跳转地址  
    output [31:0] imm_sign_ext,   // 立即数符号扩展  
    output [31:0] imm_zero_ext,   // 立即数零扩展  
  
    // ===== 控制信号输出 =====  
    output reg_write,            // 寄存器堆写使能  
    output mem_read,            // 存储器读使能  
    output mem_write,           // 存储器写使能  
    output mem_to_reg,          // 写回数据来源(0:ALU, 1:MEM)  
    output alu_src,             // ALU 源操作数选择(0:rt, 1:imm)  
    output [1:0] reg_dst,        // 写回寄存器选择(00:rt, 01:rd, 10:$31, 11:$ra)  
    output [3:0] alu_op,         // ALU 操作码  
    output [4:0] write_reg,      // 写回寄存器号  
    output sign_ext,            // 符号扩展控制(0:零扩展, 1:符号扩展)  
  
    // ===== 分支/跳转信号 =====  
    output is_branch,           // 分支指令  
    output is_jump,             // J/JAL 指令  
    output is_jr,               // JR/JALR 指令  
    output is_link,             // 链接指令(JAL/JALR)
```

```

output [2:0] branch_type,      // 分支类型

// ===== 其他控制信号 =====
output is_shift,              // 移位指令(使用 shamt)
output is_shift_v,            // 变量移位(使用 rs)
output is_mdu_op,              // 乘除法指令
output [1:0] mdu_op,           // 乘除法操作码
output is_mfhi,                // MFHI 指令
output is_mflo,                // MFLO 指令
output is_mthi,                // MTHI 指令
output is_mtlo,                // MTLO 指令
output is_mfc0,                // MFC0 指令
output is_mtc0,                // MTC0 指令
output is_eret,                // ERET 指令
output is_syscall,             // SYSCALL 指令
output is_break,               // BREAK 指令
output is_teq                  // TEQ 指令
);

```

（2） HazardUnit 模块（在后续详细说明）

负责流水线冒险处理和 ID 阶段的提前分支判断。

Load-Use 检测：当上一条指令是 Load 且目标寄存器是当前源操作数时，生成 stall。

分支判断：利用 ForwardUnit 提供的前推数据 (ID_rs_data_fwd) 提前判断 beq, bne 等分支是否成立，输出 branch_taken 信号控制 PC 跳转。

（3） ID_EX 寄存器

连接 ID 和 EX 阶段的流水线寄存器。传递控制信号（WB/MEM/EX 级控制）、操作数（rs, rt 数据）、立即数以及指令标识。在发生 stall 或异常时，具有清空（插入 NOP 气泡）的功能。他传递从寄存器堆读出的源操作数 ID_EX_rs_data 和 ID_EX_rt_data、扩展后的 32 位立即数 ID_EX_imm_ext 以及移位量 ID_EX_shamt，以及以及所有的解码控制信号包括 ALU 操作码 ID_EX_alu_op、乘除法操作码 ID_EX_mdu_op 以及针对 EX、MEM、WB 各阶段的控制位（如寄存器写使能 ID_EX_reg_write、内存读写使能 ID_EX_mem_read/write 等）。此外，它还保留了源寄存器 (rs, rt) 和目的寄存器 (rd) 的索引，供后续阶段的前推单元进行数据依赖检测。

3、EX 阶段

EX 阶段负责执行具体的算术逻辑运算或乘除法运算。该阶段包含 ALU 和 MDU 两个

核心计算部件，并集成了数据前推逻辑，用于解决数据冒险。

(1) ALU 模块：算术逻辑单元，完成加减、逻辑运算、移位、比较等操作。

根据 `alu_op` 信号对输入操作数进行计算，输出 32 位结果 `EX_alu_result`。

```
module ALU(  
    input [31:0] a,          // 操作数 1  
    input [31:0] b,          // 操作数 2  
    input [3:0] aluc,        // ALU 控制信号  
    output reg [31:0] r,     // 运算结果  
    output reg zero,         // 零标志位  
    output reg carry,        // 进位标志位  
    output reg negative,     // 负数标志位  
    output reg overflow      // 溢出标志位  
);
```

(2) MDU 模块：乘除法单元，独立处理 `mult`, `div` 等耗时指令。

内置状态机模拟乘除法周期（乘法 5 周期，除法 10 周期），通过 `busy` 信号通知流水线暂停，计算结果存入内部的 `HI/LO` 寄存器。

```
module MDU(  
    input          clk,  
    input          rst,  
    input          start,    // 启动信号  
    input [31:0] A,          // 操作数 A  
    input [31:0] B,          // 操作数 B  
    input [1:0] MDUc,        // 控制信号  
    output reg [31:0] HI,     // 高 32 位结果 / 余数  
    output reg [31:0] LO,     // 低 32 位结果 / 商  
    output reg      busy      // 忙信号  
);
```

(3) EX_MEM 寄存器：连接 EX 和 MEM 阶段的流水线寄存器。

将 ALU 计算结果、待写入内存的数据 (`rt_data`)、目标寄存器号以及相关控制信号传递到下一级。保存了 ALU 的运算结果 `EX_MEM_alu_result`（对于 `Load/Store` 指令则是访存地址）、待写入内存的数据 `EX_MEM_rt_data`（即源寄存器 `rt` 的值）以及计算确定的最终写回目标寄存器号 `EX_MEM_write_reg`。同时，它继续向下传递 MEM 和 WB 阶段所需的控制信号（如 `EX_MEM_mem_read`、`EX_MEM_reg_write`）以及异常处理相关的指令标记（如 `is_syscall`, `is_eret` 等），以便 MEM 阶段的 CP0 模块进行异常捕获。

4、MEM 阶段

MEM 阶段处理数据存储器的读写操作，并负责协处理器 CP0 的异常处理。所有

Load/Store 指令在此阶段完成访存，系统调用和异常也在该阶段被 CP0 捕获。

(1) DMEM 模块：数据存储器的读写访问。

```
module DMEM(  
    input clk,  
    input [31:0] addr,          // 字节地址  
    input [31:0] data_in,       // 写入数据  
    input we,                   // 写使能  
    input re,                   // 读使能  
    input [2:0] mem_op,         // 访存类型: 000=LW, 001=LB, 010=LBU, 011=LH,  
                                // 100=LHU, 101=SW, 110=SB, 111=SH  
    output reg [31:0] data_out  // 读出数据  
);
```

(2) CP0 模块（和上学期的实验一样，也没有处理外部中断请求）

包含 Status, Cause, EPC 等关键寄存器。当检测到 syscall, break 或 teq 异常时，记录异常原因和返回地址，并向 IF 阶段发出跳转信号。

```
module CP0(  
    input          clk,  
    input          rst,  
    // 与 CPU 指令交互的接口  
    input          mfc0,        // CPU 指令 Mfc0 (读 CP0)  
    input          mtc0,        // CPU 指令 Mtc0 (写 CP0)  
    input          eret,        // CPU 指令 ERET (异常返回)  
    // 异常处理相关接口  
    input          exception,    // 来自 CPU 的异常请求信号  
    input [4:0]     cause,       // 来自 CPU 的异常类型码 (ExcCode)  
    input          intr,        // 外部中断请求 (本次代码未处理)  
    // 数据与地址总线  
    input [31:0]    pc,         // 当前指令的地址  
    input [4:0]     Rd,         // 指定 CP0 寄存器的地址  
    input [31:0]    wdata,      // 从通用寄存器写入 CP0 的数据  
    // 输出信号  
    output [31:0]   rdata,      // 从 CP0 读出到通用寄存器的数据  
    output [31:0]   status,     // 输出 Status 寄存器的当前值  
    output reg      timer_int,  // 定时器中断信号 (本次实验未选择)  
    output [31:0]   exc_addr    // 异常入口地址  
);
```

(3) MEM_WB 寄存器

连接 MEM 和 WB 阶段的流水线寄存器。传递 ALU 结果、DMEM 读出数据、CP0 读出数据以及最终的写回控制信号。保存了来自 ALU 的计算结果 MEM_WB_alu_result、从数据存储器读取的内容 MEM_WB_mem_data 以及从 CP0 协处理器读取的数据

MEM_WB_cp0_data。此外，它还传递了写回控制信号 MEM_WB_reg_write、数据选择信号 MEM_WB_mem_to_reg 以及写回的目标寄存器号 MEM_WB_write_reg。WB 阶段将根据这些信号，从上述三个数据源中选择正确的值写入通用寄存器堆。

5、WB 阶段

WB 阶段是流水线的最后一级，负责将指令执行的结果写回到通用寄存器堆 (RegFile)。根据指令类型，写回的数据可能来自 ALU、存储器、CP0 或链接地址。

(1) 写回逻辑 (WB MUX)

多路选择器，决定最终写入寄存器的数据来源。若是 mfc0 指令，选择 CP0 数据。若是 Load 类指令 (mem_to_reg=1)，选择 DMEM 数据。否则（运算类指令），选择 ALU 计算结果（含 jal 的 PC+8 链接地址）。

```
// WB 写回
wire [31:0] WB_write_data;
assign WB_write_data = MEM_WB_is_mfc0    ? MEM_WB_cp0_data :
                        MEM_WB_mem_to_reg ? MEM_WB_mem_data :
                        MEM_WB_alu_result;
```

(2) RegFile 模块

```
module RegFile(
    input clk,           // 时钟信号
    input rst,           // 复位信号
    input ena,           // 使能信号
    input we,            // 写使能信号
    input [4:0] Rdc,      // 写目标寄存器
    input [4:0] Rsc,      // 源寄存器 1
    input [4:0] Rtc,      // 源寄存器 2
    input [31:0] Rd,       // 写入数据
    output [31:0] Rs,      // 源寄存器 1 数据
    output [31:0] Rt,      // 源寄存器 2 数据
    output [31:0] reg28    // 输出 28 号寄存器数据(gp)，用于在数码管显示
);
```

6、冒险检测单元 (HazardUnit)

冒险检测单元负责监控流水线中的各类冲突。它主要检测 Load-Use 数据冒险（上一条指令是 Load 且下一条指令立即使用该数据），一旦检测到这些无法通过前推解决的冲突，便生成 stall 信号暂停 PC 和 IF/ID 寄存器，并刷新 ID/EX 寄存器（插入气泡）。此外，该单元还负责在 ID 阶段利用前推后的数据提前判断分支条件，直接生成 branch_taken 信号。

```

// ===== Load-Use 冒险检测 =====
wire load_use_hazard = ID_EX_mem_read && (ID_EX_write_reg != 5'b0) &&
    ((ID_EX_write_reg == ID_rs) || (ID_EX_write_reg == ID_rt));

// ===== 分支数据冒险检测 =====
wire branch_hazard = ID_is_branch && (
    // ID/EX 阶段有写寄存器的指令
    (ID_EX_reg_write && (ID_EX_write_reg != 5'b0) &&
        ((ID_EX_write_reg == ID_rs) || (ID_EX_write_reg == ID_rt))) ||
    // EX/MEM 阶段有 LOAD 指令
    (EX_MEM_reg_write && (EX_MEM_write_reg != 5'b0) && EX_MEM_mem_read &&
        ((EX_MEM_write_reg == ID_rs) || (EX_MEM_write_reg == ID_rt)))
);

// ===== Stall 信号 =====
assign stall = load_use_hazard || branch_hazard;

// ===== 分支判断 =====
wire br_eq = (ID_rs_data_fwd == ID_rt_data_fwd);
wire br_gez = !ID_rs_data_fwd[31]; // rs >= 0
wire br_gtz = !ID_rs_data_fwd[31] && (ID_rs_data_fwd != 0); // rs > 0
wire br_lez = ID_rs_data_fwd[31] || (ID_rs_data_fwd == 0); // rs <= 0
wire br_ltz = ID_rs_data_fwd[31]; // rs < 0

assign branch_taken = !stall && ID_is_branch && (
    (ID_branch_type == 3'b000 && br_eq) || // BEQ
    (ID_branch_type == 3'b001 && !br_eq) || // BNE
    (ID_branch_type == 3'b010 && br_gez) || // BGEZ/BGEZAL
    (ID_branch_type == 3'b011 && br_gtz) || // BGTZ
    (ID_branch_type == 3'b100 && br_lez) || // BLEZ
    (ID_branch_type == 3'b101 && br_ltz) // BLTZ/BLTZAL
);

```

7、数据前推单元 (ForwardUnit)

数据前推单元是动态流水线的核心！它负责解决写读相关 (RAW) 数据冒险，在硬件层面实现了冲突处理，不再需要编译器优化（这也是动态和静态的本质区别）。它通过比较 ID 级和 EX 级的源寄存器号 (rs, rt) 与后续 MEM 级、WB 级的目的寄存器号 (write_reg)，判断所需数据是否还在流水线中传输。如果发现冲突，它会控制多路选择器 (MUX)，将 EX/MEM 级或 MEM/WB 级的最新结果“旁路”直接输送到 ID 级（用于分支判断）或 EX 级（用于 ALU/MDU 运算），从而避免流水线停顿。

```
// ===== ID 阶段前推控制 =====
wire [1:0] ID_forward_a, ID_forward_b;

// ID 阶段 rs 前推控制
assign ID_forward_a = (EX_MEM_reg_write && (EX_MEM_write_reg != 5'b0) &&
    (EX_MEM_write_reg == ID_rs) && !EX_MEM_mem_read) ? 2'b10 :
    (MEM_WB_reg_write && (MEM_WB_write_reg != 5'b0) &&
    (MEM_WB_write_reg == ID_rs)) ? 2'b01 : 2'b00;

// ID 阶段 rt 前推控制
assign ID_forward_b = (EX_MEM_reg_write && (EX_MEM_write_reg != 5'b0) &&
    (EX_MEM_write_reg == ID_rt) && !EX_MEM_mem_read) ? 2'b10 :
    (MEM_WB_reg_write && (MEM_WB_write_reg != 5'b0) &&
    (MEM_WB_write_reg == ID_rt)) ? 2'b01 : 2'b00;

// ID 阶段前推 MUX
assign ID_rs_data_fwd = (ID_forward_a == 2'b10) ? EX_MEM_alu_result :
    (ID_forward_a == 2'b01) ? WB_write_data :
    ID_rs_data;

assign ID_rt_data_fwd = (ID_forward_b == 2'b10) ? EX_MEM_alu_result :
    (ID_forward_b == 2'b01) ? WB_write_data :
    ID_rt_data;

// ===== EX 阶段前推控制 =====
// (与 ID 阶段类似, 代码省略了)
```

四、实验仿真过程

1、动态流水线的仿真过程

本次实验仿真时，采用第六部分的数学模型及算法程序（计算 abcd 数组的值）来作为测试程序。为了满足实验要求的波动开关暂停功能，CPU 的设计中增加了 switch[1:0]开关的设计，可以实现暂停（switch[0]: 1=暂停, 0=运行）和切换显示功能（switch[1]: 1=显示 PC, 0=显示 Reg28）。并且在 testbench 中，模拟了开关变化，验证 CPU 设计的正确性。

编写数学模型及算法程序的汇编代码，在 Mars 中编译并生成机器码（注意由于我们的 CPU 设计的 PC 值是从 0x00400000 开始，而 Mars 的 PC 值从 0 开始，所以需要修改跳转指令的机器码，比如将 08000c2a 修改为 0810002a）。随后在 Vivado 中，配置 IMEM 的 IP 核，从 coe 文件加载指令。

部分机器码：

```

1  memory_initialization_radix=16;
2  memory_initialization_vector=
3  24100000,
4  24110100,
5  24120200,
6  24130300,
7  ae000000,
8  240a0001,
9  ae2a0000,
10 24080001,
11 2409003c,
12 0109502a,
13 00085880,
14 11400028,

```

2、编写 testbanch

```

module DCPU_tb();

    // ===== 信号定义 =====
    reg clk;
    reg rst;
    reg [1:0] switch;      // 模拟板上的开关
                           // switch[0]: 1=暂停, 0=运行
                           // switch[1]: 1=显示 PC, 0=显示 Reg28

    // CPU 输出信号
    wire [31:0] PC_out;
    wire [31:0] instr_out;
    wire [31:0] reg28_out;  // 28 号寄存器值

    // 数码管信号
    wire [7:0] o_seg;
    wire [7:0] o_sel;

    // ===== 逻辑处理 =====
    // 模拟 top_board 中的逻辑
    wire ena = ~switch[0];  // switch[0]为 1 时暂停(ena=0)

    // 显示数据选择逻辑
    wire [31:0] display_data = switch[1] ? PC_out : reg28_out;

    // ===== 模块实例化 =====

    // 1. 实例化 DCPU

```

```

DCPU uut(
    .clk(clk),
    .rst(rst),
    .ena(ena),          // 连接使能信号
    .intr(1'b0),        // 暂时不测试外部中断
    .PC_out(PC_out),
    .instr_out(instr_out),
    .reg28(reg28_out)   // 连接调试端口
);

// 2. 实例化数码管驱动 (验证显示逻辑)
seg7x16 seg7_display(
    .clk(clk),
    .reset(rst),
    .cs(1'b1),          // 始终片选
    .i_data(display_data),
    .o_seg(o_seg),
    .o_sel(o_sel)
);

// ===== 时钟生成 =====
// 10ns 周期, 100MHz
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// ===== 测试流程 =====
initial begin
    $printtimescale(DCPU_tb);

    // 1. 初始化
    rst = 1;
    switch = 2'b00; // 默认运行, 显示 Reg28
    #20;
    rst = 0;
    $display("Simulation Start. CPU Running...");

    // 2. 正常运行一段时间
    #2000;
    $display("Time=%0t: CPU Running. PC=%h, Reg28=%h", $time, PC_out, reg28_out);

    // 3. 测试暂停功能 (switch[0] = 1)
    $display("Time=%0t: [Action] Switch[0] -> 1 (PAUSE)", $time);

```

```

switch[0] = 1;

#500; // 等待一段时间，观察 PC 是否变化
$display("Time=%0t: Checking Pause State. PC=%h", $time, PC_out);

// 再次检查，确认 PC 确实没变
if (PC_out === PC_out) begin // 简单的自比较，实际应比较前后时刻的值
    $display("Time=%0t: CPU is PAUSED correctly.", $time);
end

// 4. 恢复运行 (switch[0] = 0)
$display("Time=%0t: [Action] Switch[0] -> 0 (RESUME)", $time);
switch[0] = 0;

#2000;
$display("Time=%0t: CPU Resumed. PC=%h", $time, PC_out);

// 5. 测试显示切换 (switch[1] = 1, 显示 PC)
$display("Time=%0t: [Action] Switch[1] -> 1 (Show PC)", $time);
switch[1] = 1;
#100;
$display("Time=%0t: Display Data Source switched to PC. Current Display Input=%h", $time,
display_data);

// 6. 测试显示切换 (switch[1] = 0, 显示 Reg28)
$display("Time=%0t: [Action] Switch[1] -> 0 (Show Reg28)", $time);
switch[1] = 0;
#100;
$display("Time=%0t: Display Data Source switched to Reg28. Current Display Input=%h", $time,
display_data);
end

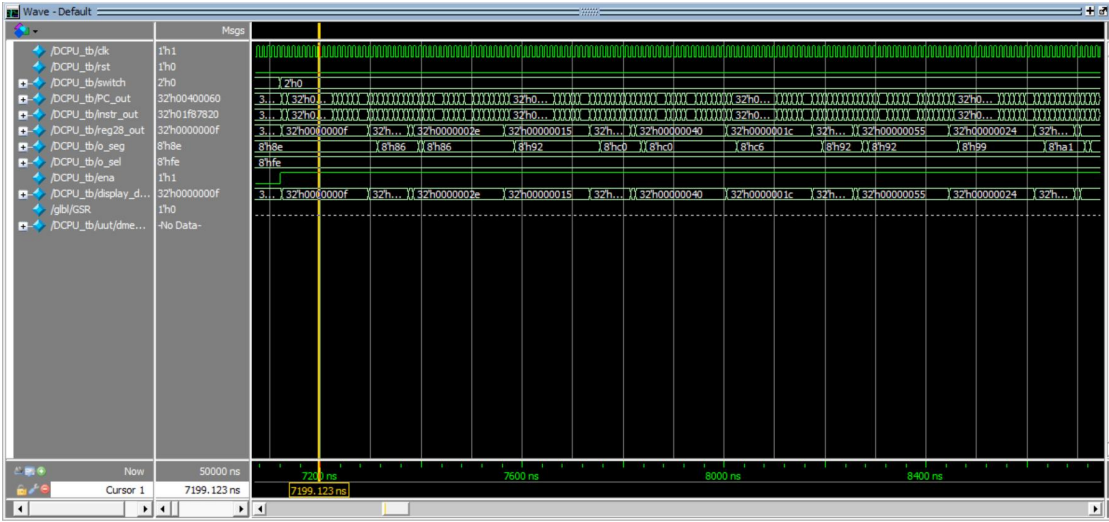
// ===== 计算值输出 =====
always @(reg28_out) begin
    if (!rst) begin
        $display("Time=%0t | Reg28 Updated: %h", $time, reg28_out);
    end
end

endmodule

```

五、实验仿真的波形图及某时刻寄存器值的物理意义

1、动态流水线的波形图和控制台输出



```
# Simulation Start. CPU Running...
# Time=265000 | Reg28 Updated: 00000001
# Time=435000 | Reg28 Updated: 00000004
# Time=515000 | Reg28 Updated: 00000001
# Time=525000 | Reg28 Updated: 00000004
# Time=705000 | Reg28 Updated: 00000003
# Time=875000 | Reg28 Updated: 0000000a
# Time=955000 | Reg28 Updated: 00000003
# Time=965000 | Reg28 Updated: 0000000a
# Time=1145000 | Reg28 Updated: 00000006
# Time=1315000 | Reg28 Updated: 00000013
# Time=1395000 | Reg28 Updated: 00000006
# Time=1405000 | Reg28 Updated: 00000013
# Time=1585000 | Reg28 Updated: 0000000a
# Time=1755000 | Reg28 Updated: 0000001f
# Time=1835000 | Reg28 Updated: 0000000a
# Time=1845000 | Reg28 Updated: 0000001f
# Time=2020000: CPU Running. PC=00400058, Reg28=0000001f
# Time=2020000: [Action] Switch[0] -> 1 (PAUSE)
# Time=7020000: Checking Pause State. PC=00400058
# Time=7120000: CPU is PAUSED correctly.
# Time=7120000: [Action] Switch[0] -> 0 (RESUME)
# Time=7125000 | Reg28 Updated: 0000000f
# Time=7295000 | Reg28 Updated: 0000002e
# Time=7375000 | Reg28 Updated: 0000000f
# Time=7385000 | Reg28 Updated: 0000002e
```

可以看到，reg28 端口的值随 CPU 运行而变化，输出顺序与 abcd 数组计算值的顺序一致——计算 a[1]=1,b[1]=4,c[1]=a[1]=1,d[1]=b[1]=4 等等，而在模拟暂停之后，reg28 的值夜相应的暂停，直到暂停结束，说明 CPU 的暂停逻辑是没有问题的。

2、结束时刻寄存器、数据存储器值的物理意义

选取结束时刻的寄存器查看值，可以看到 reg[28]中存放的是最后计算的 d[59]的值——0x9fd0b7ea。对比 Mars 运行汇编程序的寄存器和 data 的值，和仿真的结果一毛一样，证明了 CPU 设计的正确性。DMEM 的 mem 数组中，a 数组存放在 mem[0]—mem[59]，bcd 数组依次类推。查看数值，和运算结果验证，发现是一样的。查看最终 d[59]的数值，为 0x9fd0b7ea

(当然这是有溢出的, 取低 8 位的结果), 和我们用 python 程序计算的值是一样的。(python 程序源码和运行结果在后面)

The image displays a debugger interface with several panels. The top-left panel shows a list of CPU registers (R31 to R0) with their corresponding hexadecimal values. The top-right panel shows a table of registers, including Coprocessor 1 and Coprocessor 0, with columns for Name, Number, and Value. The bottom-left panel shows a list of memory segments (Data Segment) with columns for Address, Value (+0), Value (+4), Value (+8), Value (+c), Value (+10), Value (+14), Value (+18), and Value (+1c). The bottom-right panel shows a list of memory segments (Data Segment) with columns for Address, Value (+0), Value (+4), Value (+8), Value (+c), Value (+10), Value (+14), Value (+18), and Value (+1c).

Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x0000003e
\$t1	9	0x0000003c
\$t2	10	0x00000000
\$t3	11	0x000000f0
\$t4	12	0x000000e8
\$t5	13	0x0000003c
\$t6	14	0x0000006a
\$t7	15	0x000014bf
\$s0	16	0x00000000
\$s1	17	0x00000100
\$s2	18	0x00000200
\$s3	19	0x00000300
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x008f7096
\$t9	25	0x9fd0b7ea
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x9fd0b7ea
\$sp	29	0x00002ffc
\$fp	30	0x00000000
\$ra	31	0x00000000

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00000000	0x00000000	0x00000001	0x00000003	0x00000006	0x0000000a	0x0000000f	0x00000015	0x0000001c
0x00000020	0x00000024	0x00000024	0x00000037	0x00000042	0x0000004e	0x0000005b	0x00000069	0x00000078
0x00000040	0x00000080	0x00000099	0x000000ab	0x000000b6	0x000000d2	0x000000f4	0x000000f4	0x00000114
0x00000060	0x00000145	0x00000145	0x0000015f	0x00000174	0x00000196	0x000001b3	0x000001d1	0x000001f0
0x00000080	0x00000210	0x00000231	0x00000253	0x00000276	0x0000029a	0x000002bf	0x000002d5	0x0000030a
0x000000a0	0x00000334	0x00000354	0x00000387	0x000003b2	0x000003d7	0x0000040b	0x00000439	0x00000468
0x000000c0	0x00000498	0x000004e9	0x0000054b	0x0000052a	0x00000562	0x00000597	0x000005ed	0x00000604
0x000000e0	0x0000063c	0x00000675	0x000006af	0x000006a6	0x000006c0	0x000006e0	0x000006f0	0x00000700
0x00000100	0x00000004	0x00000004	0x0000000a	0x00000013	0x0000001f	0x0000002e	0x00000040	0x00000055
0x00000120	0x00000064	0x00000088	0x000000ab	0x000000c7	0x000000eb	0x00000112	0x0000013c	0x00000169
0x00000140	0x00000199	0x000001ee	0x00000202	0x0000023b	0x00000277	0x000002b6	0x000002f8	0x00000334
0x00000160	0x00000385	0x000003d0	0x00000441	0x0000046f	0x000004c3	0x0000051a	0x00000574	0x000005d1
0x00000180	0x00000631	0x00000694	0x000006fa	0x00000763	0x000007ef	0x0000083e	0x000008b0	0x00000895

六、实验验算数学模型及算法程序

1、数学模型算法程序

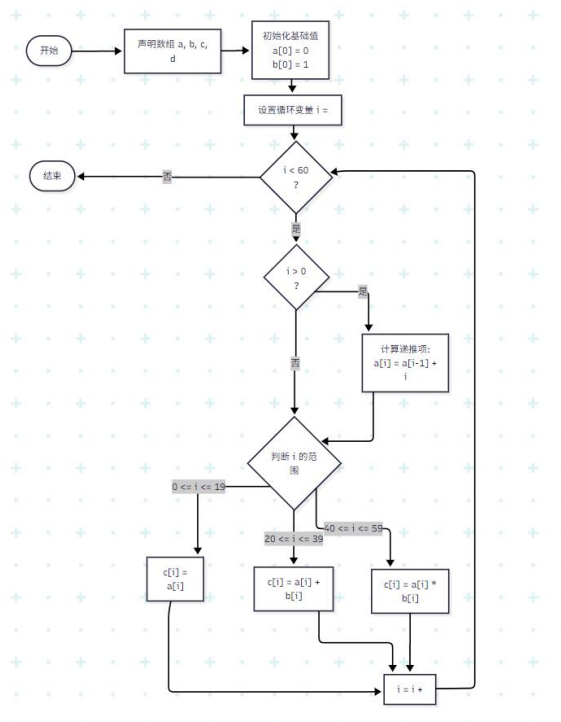
本次采用的算法程序如下所示，简单来说就是根据简单的规则递推计算 **abcd** 数组的值。

根据提议， m 为 60。根据实验模型，绘制了算法流程图。

```
int a[m],b[m],c[m],d[m];
a[0]=0;
b[0]=1;
a[i]=a[i-1]+i;
b[i]=b[i-1]+3i;

c[i]=
{
    a[i],          0≤i≤19
    a[i]+b[i],    20≤i≤39
    a[i]*b[i],    40≤i≤59
}

d[i]=
{
    b[i],          0≤i≤19
    a[i]*c[i],    20≤i≤39
    c[i]*b[i],    40≤i≤59
}
```



2、编写汇编程序

由于本次 CPU 的设计采用分支延迟槽，所以编写的汇编代码需要针对这一特性进行指令编排，将有效指令填充到延迟槽中。例如：

```
beq    $t2, $zero, end
sll    $t3, $t0, 2      # [延迟槽] offset = i * 4
```

此处利用了不跳转路径的有效指令进行填充。当循环继续时，分支不跳转，程序紧接着执行循环体。计算地址偏移量 `sll` 是循环体必须的第一步操作，将其放入延迟槽提前执行，节省了一个周期。当循环结束时，分支跳转至 `end`。延迟槽中的 `sll` 依然会执行，但计算结果不再被使用，这对程序逻辑没有影响，所以该指令可以放到延迟槽。

另外，还采用了一系列优化策略：消除 **Load-Use** 冒险——将两个数组的加载操作交错进行。先计算 **A** 的地址并加载 **A**，然后计算 **B** 的地址并加载 **B**。这样，当需要使用 **A** 的数据时，已经插入了足够的无关指令（计算 **B** 的地址和加载 **B**），从而消除了气泡；消除 **ALU-Branch** 冒险——在 `loop` 开始处，将 `sll $t3, $t0, 2` 移动到 `slt` 和 `beq` 之间，填充了气

泡。将 range1 和 range2 的判断条件（slti）提前计算，放在加载数据的间隙中。这样当执行到 bne 时，比较结果已准备好，就不需要暂停了。

```
main:
    addiu $s0, $zero, 0
    addiu $s1, $zero, 256
    addiu $s2, $zero, 512
    addiu $s3, $zero, 768
    sw     $zero, 0($s0)
    addiu $t2, $zero, 1
    sw     $t2, 0($s1)
    addiu $t0, $zero, 1
    addiu $t1, $zero, 60

loop:
    slt     $t2, $t0, $t1
    sll     $t3, $t0, 2          # 填充 slt->beq 的气泡
    beq     $t2, $zero, end
    addiu   $t4, $t3, -4         # [延迟槽] 计算 i-1 的偏移量

    # 交错加载 A[i-1]和 B[i-1]，避免 lw->use 气泡
    add     $t5, $s0, $t4        # A[i-1]地址
    lw      $t6, 0($t5)          # Load A[i-1]
    add     $t5, $s1, $t4        # B[i-1]地址
    lw      $t7, 0($t5)          # Load B[i-1]

    #提前计算分支条件，避免后面的 ALU->Branch 气泡
    slti    $k0, $t0, 20
    slti    $k1, $t0, 40

    # 计算 A[i] = A[i-1] + i
    add     $t6, $t6, $t0        # 此时距离 lw $t6 已有足够间隔
    add     $t5, $s0, $t3        # A[i]地址
    sw      $t6, 0($t5)
    addiu   $gp, $t6, 0

    # 计算 B[i] = B[i-1] + 3*i
    addiu   $t8, $zero, 3
    mult    $t0, $t8
    mflo    $t8                  # 3*i
    add     $t7, $t7, $t8        # 此时距离 lw $t7 已有足够间隔
    add     $t5, $s1, $t3        # B[i]地址
    sw      $t7, 0($t5)
    addiu   $gp, $t7, 0
```

```

# 分支逻辑优化
bne    $k0, $zero, range1
add     $t8, $t6, $t7      # [延迟槽] 预计算 c = a + b (range2 需要)

bne     $k1, $zero, range2_entry
mult    $t6, $t8           # [延迟槽] 预计算 range2 的乘法 (单周期 MDU)

# range3 (Fallthrough)
# c = a * b * b
mult    $t6, $t7
mflo    $t8
mult    $t8, $t7
j       store_cd
mflo    $t9               # [延迟槽]

range1:
    add     $t8, $zero, $t6
    j       store_cd
    add     $t9, $zero, $t7  # [延迟槽]

range2_entry:
    # range2 的乘法已经在 bne 的延迟槽中启动
    j       store_cd
    mflo    $t9             # [延迟槽] 读取乘法结果

store_cd:
    addiu   $gp, $t8, 0
    addiu   $gp, $t9, 0
    add     $t5, $s2, $t3
    sw      $t8, 0($t5)
    add     $t5, $s3, $t3
    sw      $t9, 0($t5)

    j       loop
    addiu   $t0, $t0, 1      # [延迟槽] i++

end:
    beq     $zero, $zero, end
    nop                    # 死循环

```

3. 编写 python 程序验证

按照算法逻辑，我边写了 python 程序来输出 abcd 数组最后一个值作为测试程序的基准

验证。可以看到 $d[59]=0xb9fd0b7ea$ ，截断第八位后为 $0x9fd0b7ea$ ，和我们使用 CPU 计算的
值是一致的。由于 d 的值依赖于前面所有的计算，所以可以验证所有的计算是准确无误的。

```
# 初始化
A = [0] * 60
B = [0] * 60
C = [0] * 60
D = [0] * 60

# 初始值
A[0] = 0
B[0] = 1

# 计算 A[i] 和 B[i]
for i in range(1, 60):
    A[i] = A[i-1] + i
    B[i] = B[i-1] + 3 * i

# 计算 C[i] 和 D[i]
for i in range(60):
    if 0 <= i <= 19:
        C[i] = A[i]
        D[i] = B[i]
    elif 20 <= i <= 39:
        C[i] = A[i] + B[i]
        D[i] = A[i] * C[i]
    else: # 40 <= i <= 59
        C[i] = A[i] * B[i]
        D[i] = C[i] * B[i]

# 输出结果
print(f"A[59] = {A[59]} (0x{A[59]:X})")
print(f"B[59] = {B[59]} (0x{B[59]:X})")
print(f"C[59] = {C[59]} (0x{C[59]:X})")
print(f"D[59] = {D[59]} (0x{D[59]:X})")
```

```
(base) PS D:\desktop\DB\ExCourt> python calc_d59.py
A[59] = 1770 (0x6EA)
B[59] = 5311 (0x14BF)
C[59] = 9400470 (0x8F7096)
D[59] = 49925896170 (0xB9FD0B7EA)
```

七、实验验算程序下板测试过程与实现

1、编写顶层下板模块 top_board

为了在开发板上验证 CPU 功能，编写了顶层模块 top_board。该模块负责实例化 CPU 内核、时钟分频器 (Divider)和七段数码管显示驱动 (Seg7x16)，并对接开发板上的物理外设（时钟源、复位按钮、拨动开关、七段数码管）。

```
module top_board(
    input clk,           // 100MHz 系统时钟
    input rst,           // 复位按钮
    input [1:0] switch,  // 开关: switch[0]用于暂停(1=暂停), switch[1]用于切换显示
    output [7:0] seg,     // 七段数码管段选
    output [7:0] an       // 七段数码管位选
);

// 内部信号
wire clk_cpu;           // CPU 时钟（分频后的慢时钟）
wire [31:0] instr_value; // CPU 的当前指令码
wire [31:0] pc_value;   // PC 值
wire [31:0] reg28_value; // 28 号寄存器值

// switch[0]为 1 时暂停(ena=0)，为 0 时运行(ena=1)
wire ena = ~switch[0];

// 分频器: 100MHz -> 1Hz（便于观察）
Divider divider_cpu(
    .clk(clk),
    .rst(rst),
    .clk_out(clk_cpu)
);

// DCPU 实例
DCPU cpu(
    .clk(clk_cpu),       // 使用分频后的慢时钟
    .rst(rst),
    .ena(ena),           // 暂停控制信号
    .intr(1'b0),         // 外部中断未使用
    .PC_out(pc_value),
    .instr_out(instr_value),
    .reg28(reg28_value) // 输出 28 号寄存器值用于显示
);

// 显示数据选择 switch[1]=0: 显示 reg28（运算值）; switch[1]=1: 显示 PC
wire [31:0] display_data = switch[1] ? pc_value : reg28_value;
```

```
// 七段数码管驱动
seg7x16 seg7_display(
    .clk(clk),          // 使用 100MHz 快时钟进行扫描
    .reset(rst),
    .cs(clk_cpu),
    .i_data(display_data),
    .o_seg(seg),
    .o_sel(an)
);

endmodule
```

2、设置 xdc 约束

由于这次增加了 switch[1:0]开关,需要在 xdc 文件中增加引脚约束:switch[0]映射到 J15 , 用于控制暂停。switch[1]映射到 L16, 用于切换显示内容。

同时,还需要为 CPU 添加时钟约束、输入和输出延迟。得到的时序分析报告显示时序正确。

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 1.758 ns	Worst Hold Slack (WHS): 0.113 ns	Worst Pulse Width Slack (WPWS): 4.500 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 149	Total Number of Endpoints: 149	Total Number of Endpoints: 88	
All user specified timing constraints are met.			

```
# =====
# 时钟和复位信号
# =====

set_property PACKAGE_PIN E3 [get_ports clk]
set_property PACKAGE_PIN N17 [get_ports rst]

# =====
# 开关信号 (Switch)
# switch[0]: 暂停控制 (J15, SW0) - 拨上去暂停, 拨下来运行
# switch[1]: 显示切换 (L16, SW1) - 拨上去显示 PC, 拨下来显示 Reg28
# =====

set_property PACKAGE_PIN J15 [get_ports {switch[0]}]
set_property PACKAGE_PIN L16 [get_ports {switch[1]}]

# =====
# 七段数码管段选信号 (Segment)
# =====

set_property PACKAGE_PIN T10 [get_ports {seg[0]}]
```

```
set_property PACKAGE_PIN R10 [get_ports {seg[1]}]
set_property PACKAGE_PIN K16 [get_ports {seg[2]}]
set_property PACKAGE_PIN K13 [get_ports {seg[3]}]
set_property PACKAGE_PIN P15 [get_ports {seg[4]}]
set_property PACKAGE_PIN T11 [get_ports {seg[5]}]
set_property PACKAGE_PIN L18 [get_ports {seg[6]}]
set_property PACKAGE_PIN H15 [get_ports {seg[7]}]

# =====
# 七段数码管位选信号 (Anode)
# =====

set_property PACKAGE_PIN J17 [get_ports {an[0]}]
set_property PACKAGE_PIN J18 [get_ports {an[1]}]
set_property PACKAGE_PIN T9 [get_ports {an[2]}]
set_property PACKAGE_PIN J14 [get_ports {an[3]}]
set_property PACKAGE_PIN P14 [get_ports {an[4]}]
set_property PACKAGE_PIN T14 [get_ports {an[5]}]
set_property PACKAGE_PIN K2 [get_ports {an[6]}]
set_property PACKAGE_PIN U13 [get_ports {an[7]}]

# =====
# IO 标准设置 (IOSTANDARD)
# =====

# 开关
set_property IOSTANDARD LVCMOS33 [get_ports {switch[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {switch[0]}]

# 位选
set_property IOSTANDARD LVCMOS33 [get_ports {an[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]

# 段选
set_property IOSTANDARD LVCMOS33 [get_ports {seg[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
```

```

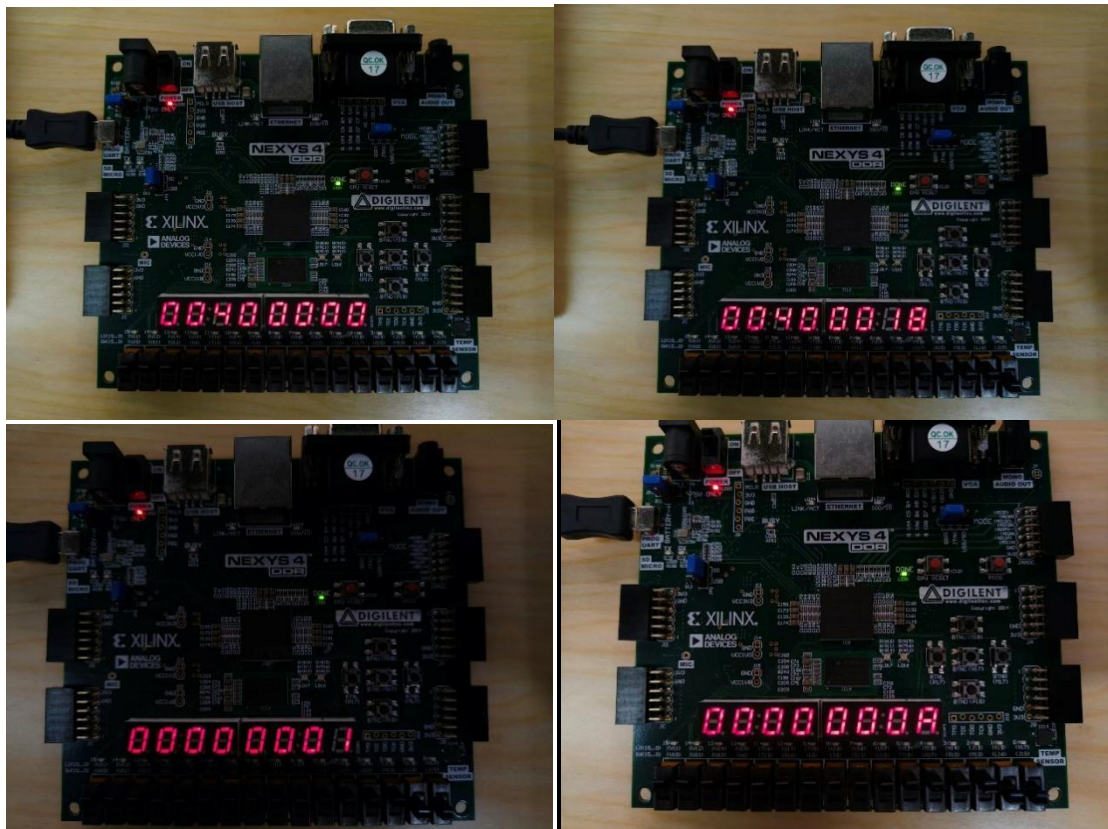
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]

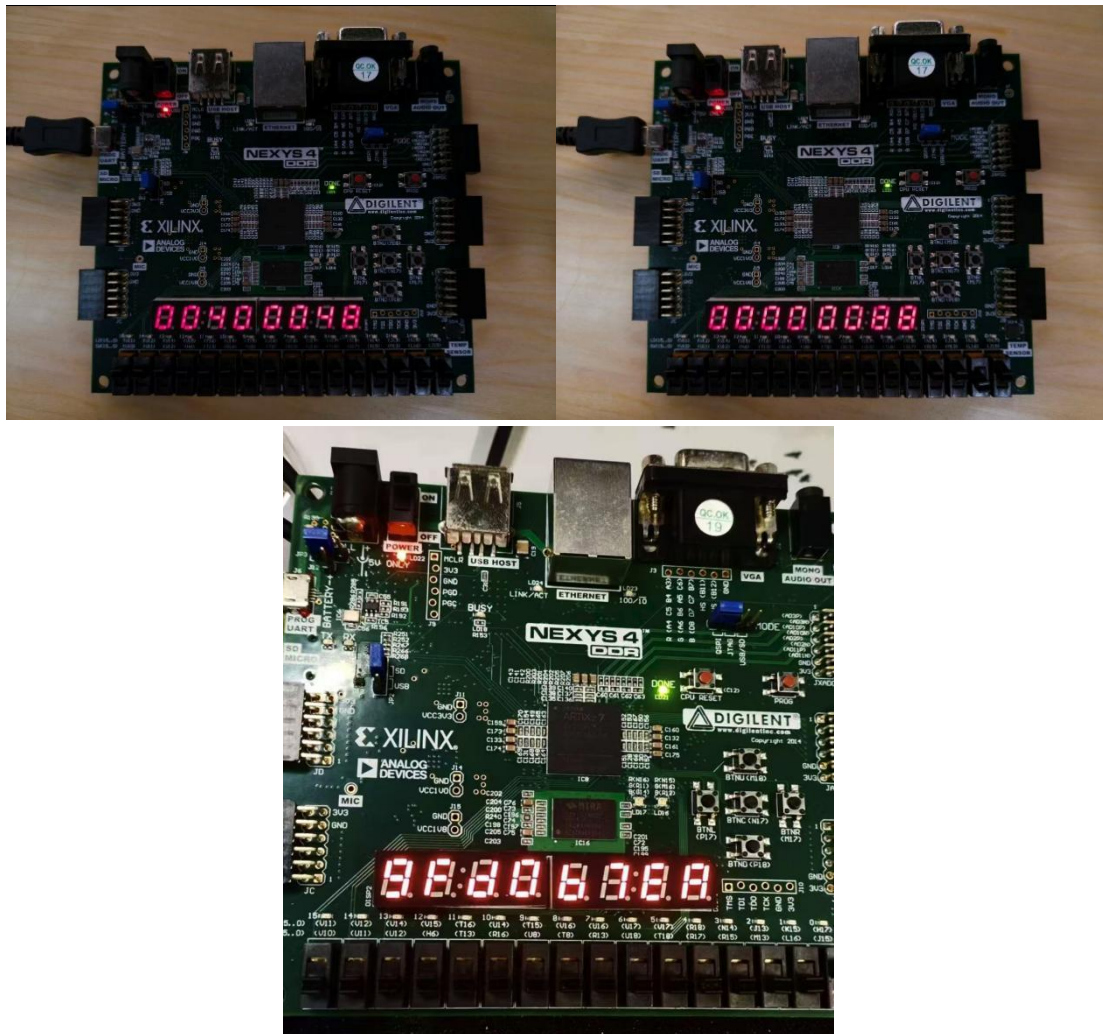
# 时钟和复位
set_property IOSTANDARD LVCMOS33 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
# =====
# 时序约束
# =====
create_clock -period 10.000 -name clk_pin -waveform {0.000 5.000} [get_ports clk]
set_input_delay -clock [get_clocks clk_pin] 8.000 [get_ports rst]
set_output_delay -clock [get_clocks clk_pin] 0.000 [get_ports -filter { NAME =~ "*" && DIRECTION
== "OUT" }]

```

3、下板验证

如图所示，刚开始 switch[0]置 1，PC 值保持在 0x00400000 不动。播下开关后，PC 值开始变化，说明指令正在执行。播下 switch[1]，切换到计算值显示模式，显示当前计算的值。再次按下暂停，PC 值和计算值都静止不动，说明实现了中断暂停执行的功能。程序执行到最后，七段数码管正确显示了 d[59]的值，为 9fd0b7ea，证明了下板过程的正确性。（最后一张是下板验收的时候拍的）





八、流水线的性能指标定性分析（包括：吞吐率、加速比、效率及相关与冲突分析）

1、吞吐率分析

吞吐率是指：CPU 在单位时间内完成的指令数，也就是总指令数/总时钟周期数。在 tb 测试文件中添加时钟周期数和指令数的计数器，可以得到：该程序执行的总时钟周期数为 2179，执行指令条数为 2119（用 Mars 统计的指令条数为 2121，因为在 Mars 中将最后的死循环指令改为 addiu+syscall 停机指令，否则死循环无法统计）。

则吞吐率可计算得到：

$$\text{吞吐率 } TP = \frac{\text{总执行指令条数}}{\text{总执行时钟周期数}} = \frac{2119}{2179} = 0.972464$$

```
// ===== 性能统计逻辑 =====
always @(posedge clk) begin
    if (rst) begin
```

```

// 复位时清零, 防止 x 传播
total_cycles <= 0;
instr_count <= 0;
end
else begin
    total_cycles <= total_cycles + 1;
    // 统计有效执行指令数 (近似: ena 有效且流水线未停顿)
    if (ena && !uut.stall) begin
        instr_count <= instr_count + 1;
    end
end
end
end

```

Detected Infinite Loop (End of Program). Stopping simulation.

=====

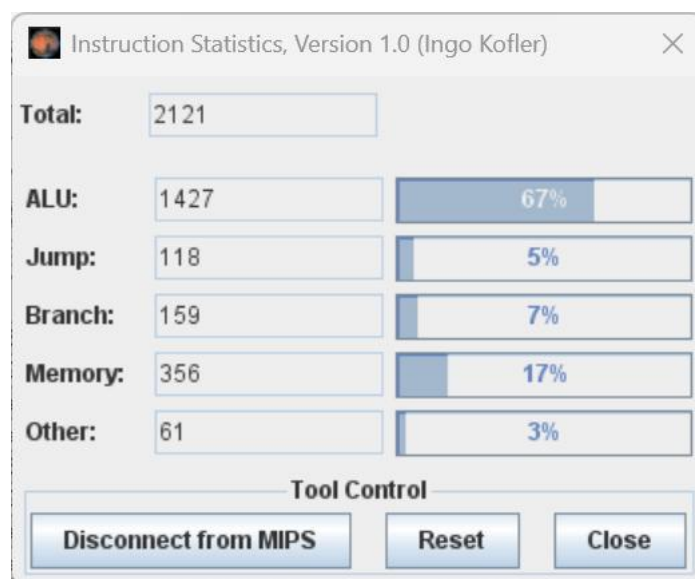
Performance Statistics:

Total Cycles: 2179

Total Instructions (approx): 2119

Throughput (IPC): 0.972464

=====



2、加速比分析

加速比是指不使用流水线的时间与使用流水线的的时间之比。将本次的流水线 CPU 与一个不使用流水线的等效多周期 CPU 的时钟周期进行比较。根据 Mars 的指令分析我们知道(去掉一条 addiu 和一条 syscall)，AIU (包括 R 型和 I 型) 指令数为 1426，所需周期数为 4；Jump 指令数 118 条，Branch 指令数为 159，所需周期数为 3；Memory 访存指令数为 356，所需周期数为 4.5 (假设 load 和 store 各占一半)；其他指令数 60，所需周期数为 4。

计算总周期数: $1426 \times 4 + 118 \times 3 + 159 \times 3 + 356 \times 4.5 + 60 \times 4 = 8377$

加速比可计算:

$$S = \frac{\text{非流水线CPU指令所需时钟周期数}}{\text{流水线CPU时钟周期数}} = \frac{8377}{2179} = 3.84442$$

3、效率分析

流水线 CPU 的效率是即流水线各级功能段的利用率。

$$E = \frac{\text{非流水线CPU指令所需时钟周期数}}{\text{流水线CPU时钟周期数} \times 5} = \frac{8377}{2179 \times 5} = 0.768885$$

4、相关与冲突分析

(1) 数据相关：当前指令依赖于前几条指令的执行结果 (RAW)

● 解决策略 1：数据前推

例如：addiu \$t4, \$t3, -4

add \$t5, \$s0, \$t4

对于 ALU-ALU 冒险（如 addiu 后跟 add），ForwardUnit 会检测到依赖关系，控制多路选择器将 EX/MEM 级或 MEM/WB 级的结果直接“旁路”送入 ALU 输入端。这实现了零气泡处理，不影响性能。

● 解决策略 2：流水线暂停

例如：lw \$t6, 0(\$t5)

add \$t6, \$t6, \$t0

对于 Load-Use 冒险（lw 后紧跟使用该数据的指令），数据必须等到 MEM 阶段结束才能读出，无法前推。HazardUnit 检测到此情况（ID_EX_mem_read 为真且寄存器号匹配）时，会拉高 stall 信号，冻结 PC 和 IF/ID 寄存器，并向 ID/EX 插入一个气泡。这会引入 1 个时钟周期的性能损失。

(2) 控制相关：分支或跳转指令改变 PC 值，导致缺省取入的指令无效

● 解决策略 1：提前分支判断

将分支判断逻辑从 EX 阶段移至 ID 阶段。利用前推后的数据在 ID 译码阶段直接计算是否跳转。这大大减少了分支预测失败时的惩罚代价。

● 解决策略 2：延迟槽

本 CPU 设计采用延迟槽技术。硬件上，当 ID 阶段确认跳转时，并不冲刷 IF 阶段已经取出的指令，而是允许其继续执行。软件上（汇编编写），通过在跳转指令后填充一条有效指令（如 i++ 或预计算指令），充分利用了这个周期，从而避免了控制冒险带来的性能损

失。而对于 `syscall`, `break` 等异常，由于无法预测，一旦 `CP0` 确认异常，必须强制刷新流水线并跳转至异常向量地址。

九、总结与体会

本次实验，我在先前五级流水线的基础上，完成了一次质的飞跃。为了支持 54 条指令，我必须处理有符号/无符号的算术扩展细节（如 `add` 与 `addu` 的溢出区别）、变长移位指令（`sliv`）、以及复杂的字节/半字访存（`lb/lhu`）。这让我深刻体会到，CPU 设计的难点往往不仅仅在于通用的数据通路，也在于如何处理那些边界情况。

这次实验相较于上次，需要增加乘法指令。所以需要引入 MDU 乘除法单元来处理乘法指令。上学期设计多周期 CPU 时我已经知道乘除法无法像 ALU 那样在单周期内完成。所以为了模拟真实硬件的耗时，为乘法和除法设计了多周期状态机，并引入了 `busy` 信号。但是，在流水线 CPU 上使用 MDU 时会带来新的结构性冒险——当流水线中的指令试图访问正在忙碌的 MDU 时，必须暂停流水线。这打破了我之前“流水线就是不停顿”的固有印象，让我学会了如何在高性能（流水线）与慢速部件（MDU）之间通过握手信号（Stall 机制）实现同步。

在最开始设计完 CPU 开始仿真 debug 的时候，遇到了一个错误。A 数组的值没问题，但是 b 数组的值整体后移了一格。具体来说，`b0=1` 没问题，但是 `b1` 本该是 4，但是还是 1；`b2` 本该是 10，但是是 4...经过检查，发现问题主要源于 HazardUnit 的检测逻辑遗漏以及 DCPU 模块中 HI/LO 寄存器的更新时机错误，导致 `mflo` 读取到了上一次乘法的结果。A 的计算不涉及乘法，所以没问题，而 b 的计算需要乘法，所以出错了。所以，设计 CPU 的时候，HazardUnit 不仅要看 MDU_busy，还要看当前 EX 阶段是否正在启动一条 MDU 指令！

在上一次实验中，遇到分支指令我通常采用简单的“预测失败+冲刷”策略。而在本次设计中，我深入实践了延迟槽技术。通过在硬件上移除对分支后一条指令的冲刷逻辑，并配合汇编代码的指令重排（将有效指令放入延迟槽），我成功避免了大量因分支跳转带来的流水线气泡。这让我深刻理解了“软硬件协同设计”——硬件提供机制（不冲刷），软件提供策略（填充延迟槽），共同来提高处理器的性能。

在本次设计中，我将原先的冒险冲突模块进行了严格的模块化解耦——ForwardUnit 专注于数据前推机制，HazardUnit 专注于冒险冲突机制，只管判断何时该停（Stall）、何时该

冲刷（Flush）。这种分离使得我在面对复杂的 Load-Use 案例和跳转指令时，能够清晰地定位问题。例如在调试 jalr 指令时，我只需在 ForwardUnit 增加通路，而无需改动 HazardUnit 的暂停逻辑，极大地降低了维护成本。

实验的最后一步——FPGA 下板验证，我也遭遇了很多困难。Vivado 综合时的各种报错（如 generated file not found、EXCEPTION_ACCESS_VIOLATION），不断的调试、修改代码、修改 tb 测试文件观察波形和输出、优化算术逻辑写法、调整时序，最终成功下板验证。尤其是综合失败，报 EXCEPTION_ACCESS_VIOLATION 错的这一步，我在网上查阅了大量资料，试了无数种方法也没解决，后来重构了 DMEM 的逻辑才得以解决。在这个过程中，不仅 debug 能力提高了，我的心态和情绪调节能力也得到了很大的锻炼。作为学计算机的牛马，在以后的科研或者工作道路上，比这复杂的多的多得多的困难也比比皆是，所以在课程中，磨练意志我认为是很有必要，也能收获很多的。

这次实验的结束，是一个五级流水线 CPU 的诞生，也是我计算机系统结构认知的大提升。我不再将 CPU 视为一个黑盒，而是看作无数个多路选择器、寄存器和状态机精妙配合的产物。那些曾经枯燥的术语——数据冒险、精确异常、延迟槽，如今都已化作我代码中的一行行逻辑，成为我未来探索更复杂系统的坚实基础。如果有机会，我还会继续探索更复杂的乱序执行 CPU 等等。

十、附件（所有程序）

本次 CPU 设计的所有源代码存放在根文件夹“CPU 设计源码”中。分为两个子文件夹：



1、CPU 源代码文件

此文件夹中包含本次 CPU 设计的 VerilogHDL 源码，包括.v 文件、tb 文件、xdc 文件和 IP 核配置文件。

注意！使用源代码下板时，需要在 Synthesis Settings 中修改-max_dsp 为 0，否则在实现环节有概率会报错！这是因为 Vivado 2016 在处理 32 位 x 32 位大位宽乘法时，试图把 4 个 DSP 小模块“串联”起来做一个大的 32 位乘法器，但在 FPGA 芯片上找不到连续的位置来放置它们，导致布局失败。



名称	修改日期	类型	大小
ALU.v	2025/12/16 21:48	V 文件	4 KB
Controller.v	2025/12/16 21:48	V 文件	14 KB
CP0.v	2025/12/15 0:32	V 文件	4 KB
DCPU.v	2025/12/17 18:59	V 文件	23 KB
Divider.v	2025/5/18 23:29	V 文件	1 KB
DMEM.v	2025/12/16 21:48	V 文件	2 KB
ForwardUnit.v	2025/12/17 19:25	V 文件	5 KB
HazardUnit.v	2025/12/17 18:59	V 文件	4 KB
IMEM.v	2025/12/16 21:49	V 文件	1 KB
MDU.v	2025/12/18 12:19	V 文件	3 KB
RegFile.v	2025/12/16 21:50	V 文件	2 KB
seg7x16.v	2025/11/15 19:00	V 文件	3 KB
top_borad.v	2025/12/16 21:50	V 文件	2 KB

2、汇编、验证文件

此文件夹中存放 python 验证程序、汇编程序和生成的机器码。

名称	修改日期	类型	大小
abcd59.py	2025/12/27 15:09	Python 源文件	1 KB
verify_optimized.asm	2025/12/18 0:16	Assembler Source	3 KB
verify_optimized3.coe	2025/12/17 19:41	COE 文件	1 KB